

Generating a Specialized *De Novo* Protein Designer with ProteinMPNN

신주영, 이동현, 오승재, 김문수

ABSTRACT

In this paper, we describe the transfer learning process of the state-of-the-art *de novo* protein designing neural network model. ProteinMPNN [1] turns into a specialized designer that generates a specific family of proteins with novel and confident protein sequences. Our research has revealed that while not training the ProteinMPNN vanilla model, training an additional softmax layer results in a that model can generate new protein sequences with high protein stability [1] compared to the ProteinMPNN vanilla model. Our novel approach to *de novo* protein design can lead to cheaper or accelerated protein functional studies in that our model can generate proteins comprised of new amino acid sequences but with the same structure overall.

CCS Concepts

- Molecular structural biology→Computational biology
- Life and medical sciences→Bioinformatics→Applied computing

Keywords

de novo protein; Protein Structure; Protein Engineering; Biotechnology; Protein Design; GNN; MPNN,

1. INTRODUCTION

Everything that lives is deeply related to proteins. All living organisms use proteins for their internal structure, actor, producer, and others. In other words, without proteins, any living organism cannot exist anymore. Proteins are core parts of all living organisms in the world. In a narrower view, the human genome codes for several ten thousand genes in their chromosome as inheritable genetic elements. Out of those proteins, some proteins get out of order in a lifetime, causing disease. The representative example for this case is cancer. Among cancer case studies, they say that if the *ras* gene malfunctions, it results in unlimited cell division and leads to cancer [5]. This degenerative deviancy results from a single-point mutation (single amino acid changes in all protein sequences). Like this, a seemingly trivial change in a protein sequence can lead to nearly unpredictable results, and this is why many researchers worldwide are continuously studying protein and their functions over time.

Protein functions are closely related to their structures [10]. Based on this knowledge, researchers have found that a specific mutation in a protein changes its functionalities completely [5]. They selected a few target protein mutations based on two major methodologies, qualitative analysis [6] and library screening [7]. The researchers finally introduce the mutant residues to a target protein to study the effect of each candidate mutation. These research methods, however, have weaknesses in that the first takes too much time for target selection, and the latter requires high economic costs. With these experimental hardships, computer-based protein structural/functional research has appeared. Generating protein sequences having desired structure, however, has been a long-standing problem in the protein research community [2]. It was previously necessary to obtain an amino acid sequence encoding the desired structures by using low-energy optimization in an energy manifold based on the laws of physics

[3]. This physics-based method, however, requires a high computational cost and has bullied protein researchers worldwide as bottleneck steps. Recently, studies applying various deep learning models, such as a fully connected layer [2] and SE(3)-Transformer [4], have been published. These researchers present various *de novo* protein designing techniques, and they have proved to be confident with numerous experimental methods such as circular theory (CD), electron microscopy (EM), and other well-established methods [1]. Among the *de novo* protein designer models, ProteinMPNN is the state-of-the-art (SOTA) model for this task. We focused on the properties of ProteinMPNN, which is the SOTA protein designer model for generic target proteins. The protein functions, however, are closely related to their structures [10], so we assume that the ProteinMPNN model should generate confident *de novo* protein designs but not the best *de novo* design. Therefore, we hypothesized that the ProteinMPNN model should be improved on their designing qualities by transfer learning (TL) on a specific protein family dataset that shares a function. We implemented various transfer learning tasks and validated *de novo* designs from each TL resulting model.

2. Methods

The research paper that performed protein sequence design using message passing neural network (MPNN), a type of graph neural network (GNN), was published in Science this September [1]. With the idea from the paper, we hypothesize that TL on the generic protein design model with the specific protein family data can generate the specialized protein designer, who can design proteins with new sequences holding the same structures. It is because protein structures and their functions are deeply connected [10]. Therefore, we assume that when the well-trained protein design model is driven to design a specific protein function, that model will make the confident protein with new sequences integrated. Based on these thoughts, we build several *de novo* protein design models, and these models can find proteins composed of new amino acid sequences with the confident structure our group desires.

Here, we show the process of various transfer learning (TL) approaches on the ProteinMPNN, the generic protein design model, with a specific protein family dataset. With these models, we design the *de novo* monomeric red fluorescent proteins (mRFP). The *de novo* designs are used for *in silico* validation to prove which TL approach is the best for giving bias to make a general model to a specialized model.

1. Dataset preparation

The dataset used in this study is from the RCSB Protein Data Bank (PDB) [8], where protein specialists worldwide deposit experimentally validated protein structure data. Many computation-aided protein predictions and designing projects have used this dataset [9], so we follow the research standards in this field and adopt it with proper data preprocessing standards. We collect the proteins belonging to the red fluorescent protein family for feeding the information of red fluorescent function to the neural network models.

2. Transfer Learning

3. Model parameter visualization over the learning process

3. Model parameter visualization over the learning process

4. Monte Carlo Cross-validation (MCCV)

4. Monte Carlo Cross-validation (MCCV)

5. In silico validation with AlphaFold v2

5. In silico validation with AlphaFold v2

3. Results

3. Results

3.1 Dataset Preparation

3.1.1 Vanilla ProteinMPNN model selection

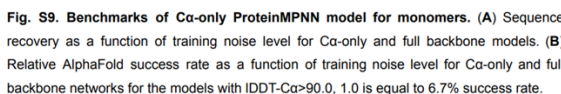


Fig. S9. Benchmarks of Ca-only ProteinMPNN model for monomers. (A) Sequence recovery as a function of training noise level for Ca-only and full backbone models. (B) Relative AlphaFold success rate as a function of training noise level for Ca-only and full backbone networks for the models with IDDT-Ca>90.0. 1.0 is equal to 6.7% success rate.

Figure 1. Benchmarks on the two types of ProteinMPN vanilla models

3.1.2 Target protein data preparation

3.1.2 Target protein data preparation

3.1.2.1 Choosing the target protein



Figure 2. The plasmid construct for expressing mRFP

3.1.2.2 Choosing the target designing region

After investigating the slightly different protein region, we found that this region is called CH6 chromophore [20] [S12]. Based on this finding, the protein region containing CH6 chromophore should be a core function part of the mRFP. Thus, we devised two

protein designing strategies; designing for all protein sequences or the functionally important region only

3.1.2.3 mRFP structural data preparation by homology modeling

For converting the protein sequence into a PDB format file that contains the information of the protein on the 3-D coordinate, we used the homology modeling technique [16]. Since the protein we want to design has not been determined experimentally yet, the 3-D model of the target protein, mRFP, was constructed and analyzed through the SWISS model. As a result of the SWISS modeling, the modeling template was the PDB entry 2qli-A chain, which has a sequence identity level of 93.72% (Fig. 3, 4). Then we evaluated this 3-D protein model. The model with the highest global quality estimation score (GMQE) of 0.94 is used for the latter protein designing steps. In general, a higher GMQE is "better", while a value above 0.7 can be considered reliable to follow the predicted quaternary structure in the modeling process [17]. Thus, the resulting mRFP model is confident enough to use for our further experiments.



Figure 3. SWISS modeling process for mRFP

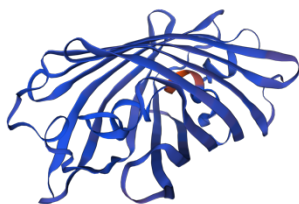


Figure 4. The resulting model of SWISS modeling

We found that the protein constructed by the SWISS model is made into a testable state and then visualized (Fig 4) [S8]. The TITLE section on the head of the file shows the SWISS model built into this file and the proper target protein name, mRFP. Then, we parse the SWISS-modeled mRFP data. We use the PDB parser in the helper functions of the public repository [1] [S9]. We adopted and adapted the Python script (parse_multiple_chains.py) used in the existing model to parse the input data we put in. After parsing, the PDB format data is converted to jsonl properly. We used this data for future design steps.

3.1.3 Querying PDB for composing RFP dataset

Using PDB query API, we fetch the protein data of the RFP family from PDB [S1]. A total of 978 proteins are in this query result. After applying a 30% identity cutoff on the query result, 314 proteins were in the final dataset. In this query, we do not include

Computed Structure Models (CSM) from PDB, because the AlphaFold should be the orthogonal control for the newly trained models, so in the TL process the CSM data must not flow into the candidate models.

3.1.3.1 Preprocessing RFP dataset

The resulting RFP dataset contains experimentally determined structures of RFPs by researchers worldwide. Because the data is from experimental settings, the single PDB data may include both RFP and other biologically significant proteins like protein kinase or RFP itself. RFP is a monomeric protein naturally [13], and our TL models should also generate monomeric RFP sequences as their output. In this situation, proteins other than monomeric RFP can act as noise, so we should preprocess each entries in the RFP dataset.

1. Applying the quality filter

The queried dataset follows the CATH4.2 dataset structure [1]. We queried protein assemblies in the PDB on Aug 02, 2021. Because the data is generated experimentally, a few PDB entries have low quality. Thus, we applied cut-off standards [1]. We use the protein data with X-ray crystallography or Cryo-EM to better than 3.5 angstroms. Datasets are separated using entity separation(8:1:1 for training: validation: testing) but not clustered separation [1]. To apply the protein entity-based dataset preparation, we modified the list of metadata, List.csv properly [S10]. The resulting single chain-based training dataset size is 1079.

2. Splitting each protein complex data into single chains

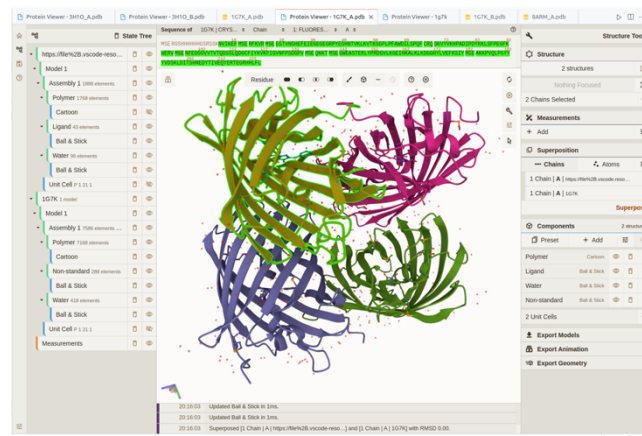


Figure 7. Overall structure of a multichain protein (PDB id:1G7K)

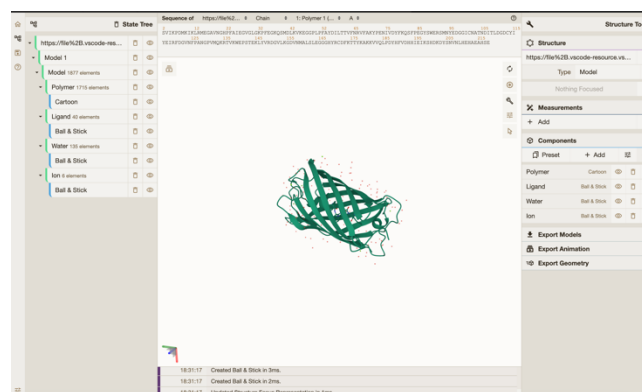


Figure 8. The visualization of a split protein data (file name: 2GX2_A)

There are many multichain protein data in the RFP dataset (Fig. 7). After the exhaustive dataset exploration process, we found three types of multichain data. The first type is multiple RFPs in a single data entry [S2]. The second type is RFP as a tagging protein, which is adjacent to biologically significant proteins [S3]. And the last is just an unrelated protein [S5, S6]. Thus, we should split these complex data into single chains. We implement this process with our custom Python script using the Biopython library [14]. The split chain data are stored in a local disk annotated the file name with a chain name and separated by a ‘_’ character (Fig. 8). For validating split results, we manually superpose split protein chain on the originating protein file. The superposing result shows root mean square deviation (RMSD) as 0 angstrom, meaning that no deviancy in this process is detected (Fig. 7).

Figure 9. The header description of the split RFP proteins

Figure 10. The header description of unrelated proteins

Figure 11. The protein type distribution of the RFP dataset (Left: calculation based on protein entries. Right: calculation based on the split chain entries)

Pseudocode for the encoder layer (V - node features, E - edge features):

```
def encoder_layer_forward(V, E):
    M_ij = MLP[V_i, V_j, E_ij]
    dV_ij = Sum_j [M_ij]
    V_i = LayerNorm[V_i + Dropout(dV_ij)]
    dV_ij = FeedForward[V_i]
    V_i = LayerNorm[V_i + Dropout(dV_ij)]
    dE_ij = MLP[V_i, V_j, E_ij]
    E_ij = LayerNorm[E_ij + Dropout(dE_ij)]
    return V, E
```

Figure 13. Pseudo code for encoder layer

```
def decoder_layer_forward(V, E, S, mask):
    E_ij = Concat[E_ij, S_ij] * mask_ij + Concat[E_ij, 0.0*S_ij] * (1-mask_ij)
    M_ij = MLP[V_i, V_j, E_ij]
    dV_ij = Sum_j [M_ij]
    V_i = LayerNorm[V_i + Dropout(dV_ij)]
    dV_ij = FeedForward[V_i]
    V_i = LayerNorm[V_i + Dropout(dV_ij)]
    return V
```

Figure 14. Pseudocode for decoder layer

3.2.2 Transfer Learning with the three approaches

3.2.2.1 Brief introduction to TL implementation

```
model = ProteinMPNN(
    node_features=args.hidden_dim,
    edge_features=args.hidden_dim,
    hidden_dim=args.hidden_dim,
    num_encoder_layers=args.num_encoder_layers,
    num_decoder_layers=args.num_decoder_layers,
    k_neighbors=args.num_neighbors,
    dropout=args.dropout,
    augment_eps=args.backbone_noise,
)
model.to(device)

if args.fixed_network_transfer_learning:
    model_final_classifier = ProteinMPNN_FixedRFP(
        hidden_dim=args.hidden_dim,
        dropout=args.dropout,
    )
```

Figure 15. The code snippet of TL.

We use a ProteinMPNN model as a pre-trained model [1]. We aim to apply TL (transfer learning) to this pre-trained model to create the De Novo Protein designer model with the best performance. To find the best TL method for our purpose, we implemented the three TL scenarios that changed the weights of probability extractors. The baseline weight of the pre-trained model is v 48 020.pt which is in the public GitHub repository [1]. The core code for TL implementation is depicted above (Fig. 15). Our three TL scenarios below are devised from a computer vision tutorial [21]. We made two major TL scenarios, Finetuning the network and the network as a fixed feature extractor. We termed the fine-tuning as scenario 1. For the fixed feature extractor, we hypothesized the two TL scenarios, which are scenario 2 and scenario 3. Scenario 3 follows the major TL trend, but we put some novelty on scenario 2 as we fixed the whole network and

placed an additional softmax layer. Further descriptions are in each section.

3.2.2.2 Scenario 1: Update on the whole network.

In this scenario, instead of random initialization, we initialize the network with a pretrained network, ProteinMPNN like the convnet that is trained on the imagenet 1000 dataset. The rest of the training looks as usual [21]. The whole neural network model is open to gradient flows and learns on the RFP dataset.

3.2.2.3 Scenario 2: Fixed network then update additional network. [Frozen-TYPE1]

```
You, 56 minutes ago | 1 author (You)
class ProteinMPNN_FixedRFP(nn.Module):
    def __init__(
        self,
        num_letters=21,
        hidden_dim=128,
        dropout=0.1,
        scale=30
    ):
        super(ProteinMPNN_FixedRFP, self).__init__()
        self.scale = scale

        """ Simple MLP for RFP de novo design """
        self.W_in = nn.Linear(num_letters, hidden_dim, bias=True)
        self.W_out = nn.Linear(hidden_dim, num_letters, bias=True)

        self.dropout1 = nn.Dropout(dropout)
        self.norm1 = nn.LayerNorm(hidden_dim)
        self.act = torch.nn.GELU()

        for p in self.parameters():
            if p.dim() > 1:
                nn.init.xavier_uniform_(p)

    def forward(self, log_probs):
        logits = self.W_out(self.norm1(self.dropout1(self.act(self.W_in(log_probs)))))
        final_log_probs = F.log_softmax(logits, dim=-1)

        return final_log_probs
```

Figure 16. Code that shows output (log probabilities) from the MLP layer added to ProteinMPNN model.

In this scenario, we fix the whole network and then update the additional network (Fig. 16). The newly added MLP layer is the only part where the model learns. The weights of the ProteinMPNN model are fixed at first (Fig. 17). We hypothesized this TL approach is probabilistically equivalent to the original fixed feature extractor approach, so we did not apply any other processing.

```
# fixed network TL
if args.fixed_network_transfer_learning:
    for param in model.parameters():
        param.requires_grad = False
```

Figure 17. Code snippet showing the fixing weights

3.2.2.4 Scenario 3. Fixed network, but the last layer is open to train [Frozen-TYPE2]

```
elif args.train_only_last_layer:
    for param in model.parameters():
        param.requires_grad = False

    # enable grad for the last layer + last decoder FF layer
    for param in model.W_out.parameters():
        param.requires_grad = True
    for param in model.decoder_layers[-1].dense.W_out.parameters():
        param.requires_grad = True
```

Figure 18. Code that shows how the information changed its weight in Scenario 3

In the last scenario, we freeze but re-train the last softmax layer. Like Frozen-TYPE1, one MLP layer that extracts log probabilities is under training but the softmax layer is in the ProteinMPNN model (Fig. 18). This scenario follows the original TL approach, the fixed feature extractor [21].

3.3 Model parameter visualization over the learning process

Over the training process, we visualize the training target neural network layers for each scenario. For scenario 2 and scenario 3, we visualized the softmax layer. Scenario 1, however, has much wider training candidates so we visualized only the output layers of the last encoder and the last decoder to interpret the model training process. We implement this section using tensorboard API.

3.3.1 Scenario 1: Update on the whole network.

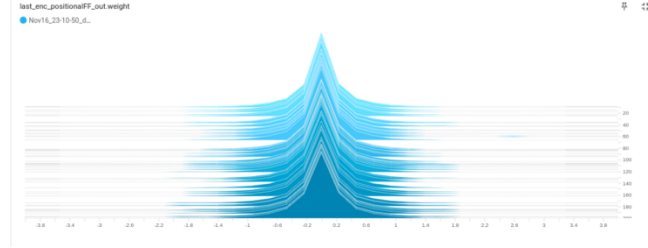


Figure 19. The weight distribution of the output layer of the last encoder over 200 epochs

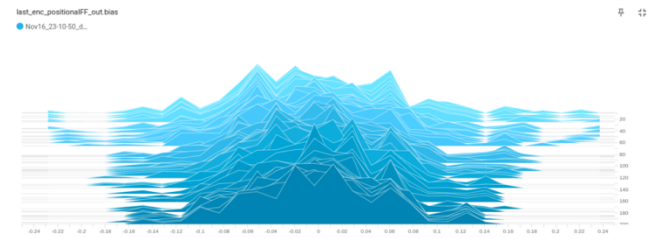


Figure 20. The bias distribution of the output layer of the last encoder over 200 epochs

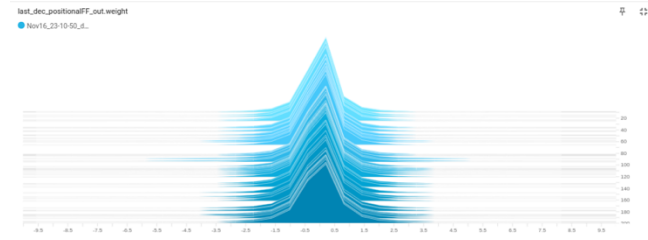


Figure 21. The weight distribution of the output layer of the last decoder over 200 epochs

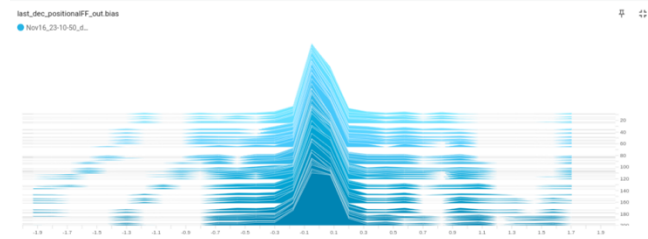


Figure 22. The bias distribution of the output layer of the last decoder over 200 epochs

Interestingly, training with the scenario 1 principle shows fixed weights but changing biases (Fig. 19, 20, 21, 22). Though all the model components are open to learning RFP data, the model has fixed weight distribution over all training steps. The difference is the changing bias distribution. The visualization shows the model

is continuously biased as we expected. The distributions on the last encoder were significant, but that of the last decoder showed not many changes.

3.3.2 Scenario 2: Fixed network then update additional network. [Frozen-TYPE1]

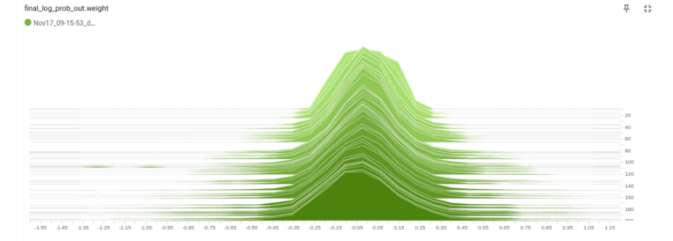


Figure 23. The weight distribution of the additional softmax layer over 200 epochs

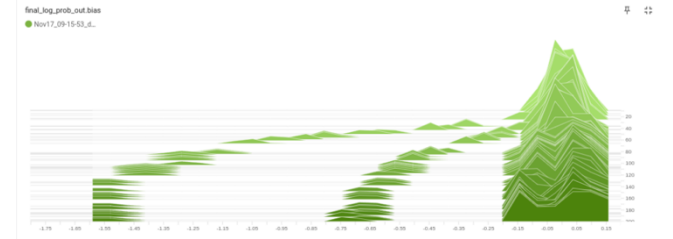


Figure 24. The bias distribution of the additional softmax layer over 200 epochs

We initialized the additional layer with Xavier initialization [1], so the initial weight distribution shows a bell shape (Fig. 23). In this scenario, weight and bias distributions are under change; weight variance goes too large, and from the bias distribution histogram, we find two islands of biases go in a negative direction as the training epoch increases (Fig. 24). We assume that the two islands of features are negatively related to RFP.

3.3.3 Scenario 3. Fixed network, but the last layer is open to train [Frozen-TYPE2]

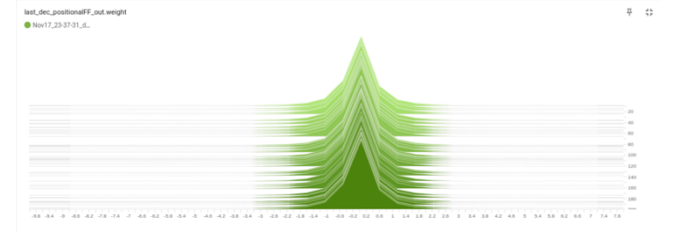


Figure 25. The weight distribution of the output layer of the last decoder over 200 epochs

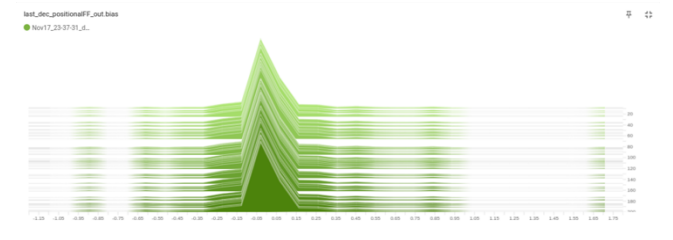


Figure 26. The bias distribution of the output layer of the last decoder over 200 epochs

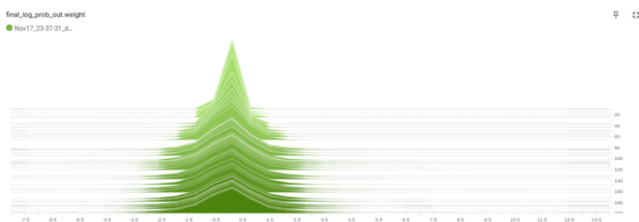


Figure 27. The weight distribution of the last softmax layer over 200 epochs

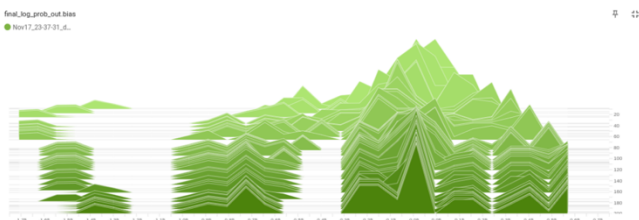


Figure 28. The bias distribution of the last softmax layer over 200 epochs

As we have shown in scenario 1, the weight and bias distributions on the last decoder seem still (Fig. 21, 22). Furthermore, we took a preliminary experiment to determine the range of learning layers. As a result, flowing gradients only to the last softmax layer show no changes from the vanilla model. Thus, we set the scenario 3 training environment as open to errors to one more layer, the softmax layer of the last decoder (Fig. 25, 26). The changes in the weight and bias distribution of the last softmax layer were quite interesting (Fig. 27, 28). The changing speed and its' form are extreme. Thus, we should validate using a learning curve of cross-validation to interpret what it means.

3.4 Monte Carlo Cross-validation (MCCV)

3.4.1 Validation settings

```
for e in range(args.num_epochs):
    # for MC CV
    train, valid = build_cv_clusters(train_original, args.debug)

    train_set = PDB_dataset(list(train.keys()), loader_pdb, train, params)
    train_loader = torch.utils.data.DataLoader(
        train_set, worker_init_fn=worker_init_fn, **LOAD_PARAM
    )
    valid_set = PDB_dataset(list(valid.keys()), loader_pdb, valid, params)
    valid_loader = torch.utils.data.DataLoader(
        valid_set, worker_init_fn=worker_init_fn, **LOAD_PARAM
    )
```

Figure 29. Code snippet for MCCV implementation

For tracking the model training process, we implemented Monte Carlo cross-validation (MCCV) (Fig. 29). We need to check whether the model is properly biased to a designer for the RFP family. With MCCV, each data is an independent entity and is tested many times through the training and validation iteration. Also, MCCV has the characteristics of high bias with low variance. Those properties are what we want a TL model to have. The representative model weights are selected when the MCCV score is saturated.

3.4.2 Drawing learning curves for each TL scenario

We used perplexity and accuracy to determine the maximum epoch number for drawing an informative learning curve as many epochs [1]. We trained scenario 1 for this task and found that each iteration contains 53 steps. The model performance saturates around epoch 200, so we fix it for TL processes [S15].

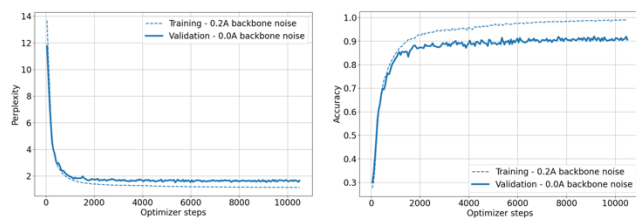


Figure 30. The learning curve of scenario 1

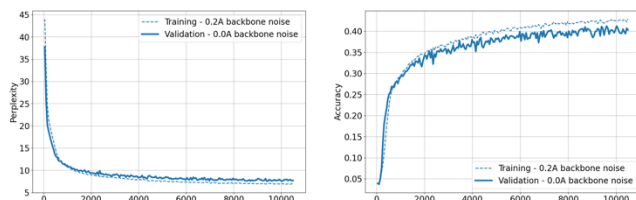


Figure 31. The learning curve of scenario 2

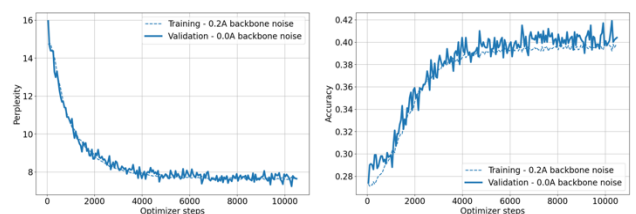


Figure 32. The learning curve of scenario 3

We choose the parameters epoch 30, 80, and 100 for scenarios 1, 2, and 3 each as an mRFP designer model. For scenario 1, the perplexity level is around 2.0, which is half of the vanilla model (Fig. 30) [S16]. The TL model from scenario 1 seems more confident on the RFP dataset as it shows a smooth learning curve. It would result from the stable parameters we showed above (Fig. 21, 22).

Scenario 2 shows a smooth curve as scenario 1, but with doubled perplexity (Fig. 31). The model of scenario 2 saturates around perplexity 8, which is nearly double the vanilla. Based on this clue, we infer the model composes de novo mRFP with its structure but with new sequence compositions.

Lastly, scenario 3 shows similar trends as it showed in the parameter histograms. The learning curve fluctuates over the epochs and is saturated around perplexity 8, which is a similar level to scenario 2. Still, we cannot figure out the meaning of fluctuating graphs. This question is solved in the following section.

3.5 In silico validation with AlphaFold v2

3.5.1 Introduction to in silico validation methods

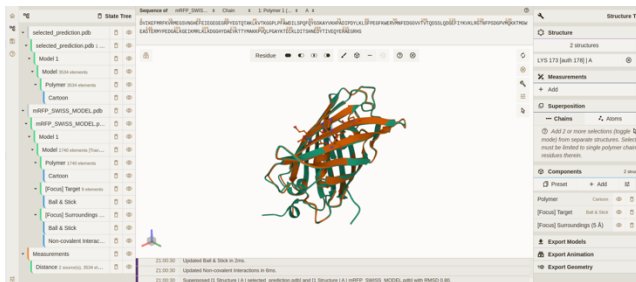


Figure 33. Superposed SWISS and AlphaFold models for mRFP

Before using the AlphaFold models, we should check if there is any structural difference between the SWISS mRFP model and

that of the AlphaFold model. Because the SWISS model would be the input structure for our protein designer models, and the output sequences following that structure will be interpreted using AlphaFold. If there is any difference between the two models for the original mRFP, then the possible structural changes cannot be attributed to the sequence composition change. Thus, we compared the SWISS and the AlphaFold protein model for mRFP. The measured root-mean-square deviation (RMSD) was 0.80 angstrom (Fig. 33), and it is an excellent level of RMSD for protein structure comparison, which is under the experimental error [23]. In our experiment, using the input structural data via homology modeling does not hamper our assumption that adopting the AlphaFold protein prediction network could be our orthogonal control network model.

For upcoming in silico validation using AlphaFold v2 [9], we designed 10 de novo protein sequences for each TL scenario. Here, we applied two design strategies for the three TL scenarios as we discussed in section 3.1.2.2 [S12, 13, 17]. We designed mRFP in two ways, generating the sequences of CH6 motif only and whole mRFP sequences. To evaluate each design, the 3-D model of the de novo sequence is predicted via AlphaFold. Then we post-process these results using the "AlphaFold analyzer" Python script to evaluate how confident each de novo protein is. The metric for determining structural confidence is pLDDT scores over mRFP residues [9]. In the following section, the left plot shows the level of the structure confidence for designing all protein sequences, while the right is for doing only the CH6 motif region.

3.5.2 Visualizing structural confidence levels of de novo protein designs

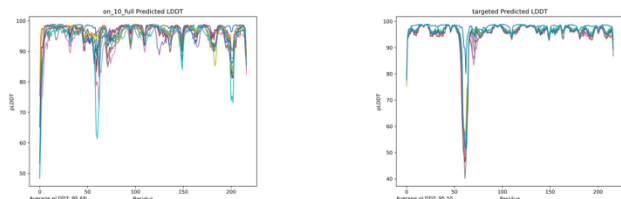


Figure 34. Overlapped pLDDT plots of vanilla model over 10 designed proteins

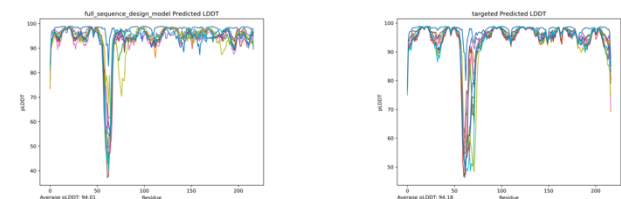


Figure 35. Overlapped pLDDT plots of scenario 1 model over 10 designed proteins

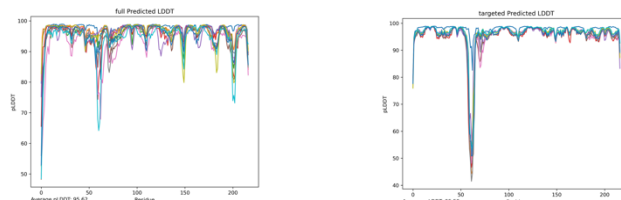


Figure 36. Overlapped pLDDT plots of scenario 2 model over 10 designed proteins

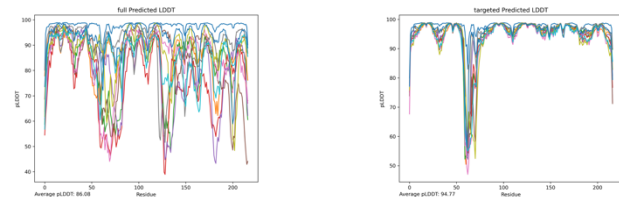


Figure 37. Overlapped pLDDT plots of scenario 3 model over 10 designed proteins

There are eight overlapped line plots. They are the structural confidence levels of the protein designs made by each model we prepared (Fig. 34, 35, 36, 37) [9]. We found two interesting points on the plots.

First, as opposed to our assumption, scenario 1 and scenario 3 shows lower overall confidence than the vanilla model on the region containing the CH6 motif (the dip around the residue number 50). At this time, the fluctuating learning curves of scenario 3 are explained. Like the learning curve, the confidence level of protein designs that the scenario 3 TL model made fluctuates (Fig. 37). We determined that scenario 3 is inappropriate for this task, as its' protein designs lose overall confidence. Meanwhile, the scenario 1 model seems confident from learning curves, but it shows more confidence loss of the CH6 region than the vanilla model (Fig. 35). It was quite a surprise for us because we thought that the learning curves are directly proportional to confidence levels in protein designs. The reality, however, is the opposite as the model generates less confident mRFP designs for each iteration.

Second, when the protein model was designed for the CH6 chromophore region only, the confidence level of that region is rather lowered (Fig. 34, 36). This is attributed to the factor considering the molecular interactions. Protein residues interact with other residues through a hydrogen bond, ionic bond, Van der Waals, forces, etc. By designing the whole protein sequences, the model could recognize the sequence level of residual interactions. Designing only the target CH6 region, however, the model takes myopic approaches, so the resulting designs are without considering the interactions between the other protein residues and itself at all. Therefore, not confident designs are generated. Recently, the research for embedding molecular interactions to the AlphaFold dataset is published and it might be a cure for settle down our myopic designing problem [22].

3.5.3 The best TL scenario for making a specialized protein designer

	swiss	sample1	sample2	sample3	sample4	sample5	sample6	sample7	sample8	sample9	sample10	average
Scenario 1 - full sequence	97.72	94.20	94.04	94.74	92.71	92.25	94.81	93.52	94.05	92.97	93.08	94.01
Scenario 1 - target sequence	97.62	94.12	92.46	94.77	95.44	94.70	93.13	92.60	92.85	93.50	94.79	94.18
Scenario 2 - full sequence	97.75	97.06	95.66	96.36	94.70	93.49	95.49	96.18	93.99	95.54	95.58	95.62
Scenario 2 - target sequence	97.75	95.28	94.77	95.51	94.79	95.25	94.90	95.53	96.06	95.24	96.05	95.56
Scenario 3 - full sequence	97.74	86.01	76.59	80.66	83.57	84.80	88.72	88.21	85.70	92.67	82.26	86.08
Scenario 3 - target sequence	97.75	95.24	94.52	94.61	93.56	93.10	96.08	93.69	93.56	95.51	94.43	94.73
Vanilla - full sequence	97.72	97.01	96.18	96.36	94.73	93.91	95.52	96.19	93.98	95.69	95.24	95.68
Vanilla - target sequence	97.68	95.36	94.79	95.21	94.67	95.32	94.83	95.94	96.03	95.26	95.93	95.55

Figure 38. Tabular description of pLDDT scores (Green: the design with the best average pLDDT for each TL model)

In the MCCV, scenario 1 has the half perplexity level of scenario 2. We assume that the doubled perplexity of scenario 2 may suggest more potential to design the given protein structure with new sequence compositions. After the AlphaFold protein confidence testing, it turns out that the higher perplexity of scenario 2 makes the model design more confident de novo

proteins than scenario 1 (Fig. 38). Blue box on the table shows the average pLDDT overall protein residues from the best design of scenario 1 model. It was lower than the best designs of scenario 2 and the vanilla model (red line, green filled boxes). That is, the model after TL scenario 1 shows no improvement in structural confidence.

Description	Scientific Name	Max Score	Total Score	Query Cover	E value	Per. Ident	Acc. Len	Accession
Chain A_PamCherry1 protein [Discosoma sp.]	Discoso...	144	144	98%	6e-39	38.50%	234	3KCS_A
Chain A_Fast Fluorescent Timer Fast-FT [Discosoma sp.]	Discoso...	142	142	93%	3e-38	38.42%	230	3LF3_A
mScarlet [synthetic construct]	synthetic...	135	135	93%	2e-35	40.59%	232	APD76535.1
mScarlet-H [synthetic construct]	synthetic...	134	134	93%	8e-35	40.59%	232	APD76537.1
Chain A_mScarlet [Discosoma sp.]	Discoso...	133	133	93%	1e-34	41.09%	217	5LK4_A
mScarlet-L [synthetic construct]	synthetic...	133	133	93%	2e-34	40.10%	232	APD76536.1
CaScarlet [Cloning Vector pSF2A-mScarlet]	Cloning...	133	133	93%	2e-34	40.10%	231	Q8H90981.1
Chain A_LSSmScarlet - Genetically Encoded Red Fluor...	synthetic...	134	134	93%	2e-34	39.60%	274	7QIN_A
mScarlet-L-3xFLAG [Cloning vector pL162-mScarlet-L]	Cloning...	133	133	93%	4e-34	40.10%	257	QTE34485.1

Figure 39. BLASTP result of the designed mRFP by TL scenario 2

The last standing model in our study, TL scenario 2, shows a similar level of structural confidence to the vanilla model, which is around 95 on average. The interesting point of this result is that the designs from the TL scenario 2 model achieve comparable confidence levels with nearly doubled perplexity levels (S16). It means the mRFP designs generated by TL scenario 2 can achieve the structural confidence level of the SOTA model with a more diversified protein sequence composition, which is the ultimate goal of the de novo protein design. We BLASTP-ed the best design from TL scenario 2, Scenario 2-full sequence/smapple1 (Fig. 39), and it outputs the best entry with E-value 6e-39 [24]. Scenario 2 proves its capability to design de novo sequences with confidence as high as the SOTA model and more potential for diversity, which can boost up many protein-related works worldwide.

4. Limitations of the study

In this study, we sought to train a protein designer model that designs a more confident protein of a specific function. There, however, are a few limitations we faced.

First, our model makes proteins that do not exist in nature, so the designs the model generates are not present in nature. We, however, still evaluate the de novo designs with the prediction model trained on the already existing proteins [9]. It directly suggests that the validation metrics we used, MCCV (perplexity and accuracy) and AlphaFold confidence level, already includes limitation. Thus, it is hard to genuinely evaluate in silico only whether the transfer learned model designs de novo protein well, as the related work writes [1]. Therefore, this study needs further validation with the conventional protein studying methods like CD, EM, and other approaches, after expressing the protein on the protein expression system.

Secondly, the time given for experiments is short to generate enough experimental data. For a high-quality comparison of the design performance variation in the various TL approaches, we need to evaluate it with at least a few hundred design data from each TL model [1]. In this study, however, the *in silico* validation method, AlphaFold, queries a single protein sequence on the 8.8 TB database, which is the significant lagging phase (~ 1hr per design) in our experimental setup. We had about two weeks to test

the model performance, so we cannot help but create only 10 mRFP designs per each TL scenario. Because of the small number of data itself, the results of the evaluation are less accurate and even misdirecting. We recognize this problem, and we will make improvements to it in no time. Then, we will get a much more accurate intuition in finding the best protein designer model for a specific function.

5. ACKNOWLEDGEMENTS

All the computing resources for this study is from Sungkyunkwan University Genome Editing Laboratory (SKKUGE).

6. Reference

- [1] Dauparas, J., Anishchenko, I., Bennett, N., Bai, H., Ragotte, R. J., Milles, L. F., Wicky, B. I., Courbet, A., de Haas, R. J., Bethel, N., Leung, P. J., Huddy, T. F., Pellock, S., Tischer, D., Chan, F., Koepnick, B., Nguyen, H., Kang, A., Sankaran, B., ... Baker, D. (2022). Robust deep learning-based protein sequence design using proteinmpnn. *Science*, 378(6615), 49–56. <https://doi.org/10.1126/science.add2187>
- [2] Anishchenko, I., Pellock, S.J., Chidyausiku, T.M. et al. De novo protein design by deep network hallucination. *Nature* 600, 547–552 (2021). <https://doi.org/10.1038/s41586-021-04184-w>
- [3] Leman, J.K., Weitzner, B.D., Lewis, S.M. et al. Macromolecular modeling and design in Rosetta: recent methods and frameworks. *Nat Methods* 17, 665–680 (2020). <https://doi.org/10.1038/s41592-020-0848-2>
- [4] Baek, M., DiMaio, F., Anishchenko, I., Dauparas, J., Ovchinnikov, S., Lee, G. R., Wang, J., Cong, Q., Kinch, L. N., Schaeffer, R. D., Millán, C., Park, H., Adams, C., Glassman, C. R., DeGiovanni, A., Pereira, J. H., Rodrigues, A. V., van Dijk, A. A., Ebrecht, A. C., ... Baker, D. (2021). Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557), 871–876. <https://doi.org/10.1126/science.abj8754>
- [5] Liu, J., Kang, R. & Tang, D. The KRAS-G12C inhibitor: activity and resistance. *Cancer Gene Ther* 29, 875–878 (2022). <https://doi.org/10.1038/s41417-021-00383-9>
- [6] Weiss, G. A., Watanabe, C. K., Zhong, A., Goddard, A., & Sidhu, S. S. (2000). Rapid mapping of protein functional epitopes by combinatorial alanine scanning. *Proceedings of the National Academy of Sciences*, 97(16), 8950-8954. <https://doi.org/doi:10.1073/pnas.160252097>
- [7] Screening Libraries To Identify Proteins with Desired Binding Activities Using a Split-GFP Reassembly Assay Meredith E. Jackrel, Aitziber L. Cortajarena, Tina Y. Liu, and Lynne Regan *ACS Chemical Biology* 2010 5 (6), 553-562 DOI: 10.1021/cb900272j
- [8] Stephen K Burley, Helen M Berman, Charmi Bhikadiya, Chunxiao Bi, Li Chen, Luigi Di Costanzo, Cole Christie, Ken Dalenberg, Jose M Duarte, Shuchismita Dutta, Zukang Feng, Sutapa Ghosh, David S Goodsell, Rachel K Green, Vladimir Guranović, Dmytro Guzenko, Brian P Hudson, Tara Kalro, Yuhe Liang, Robert Lowe, Harry Namkoong, Ezra Peisach, Irina Periskova, Andreas Prlić, Chris Randle, Alexander Rose, Peter Rose, Raul Sala, Monica Sekharan, Chenghua Shao, Lihua Tan, Yi-Ping Tao, Yana Valasatava, Maria Voigt, John Westbrook, Jesse Woo, Huanwang Yang, Jasmine Young, Marina Zhuravleva, Christine Zardecki, RCSB Protein Data Bank: biological macromolecular structures enabling research and education in fundamental